

宁波大学
NINGBO UNIVERSITY

信息安全技术实验报告

[对称加密算法实验]

姓名：	顾凯琪
学号：	226001624
课程名称：	信息安全技术
所在学院：	信息科学与工程学院
专业班级：	22 级计算机 2 班
任课教师：	徐博文 严迪群

实验项目名称	实验一 对称加密算法实验

一、实验目的与要求：

1. 熟悉 CrypTool 的基本功能；
2. 通过 CrypTool 的 Wizard 功能，完成至少 4 种对称加密算法的加解密过程。（古典 1、2）现代（1、2）
3. 明文需要包含自己的学号或姓名；
4. 尝试用 Graphical Editor 实现 Caesar 密码（凯撒密码）的加解密。

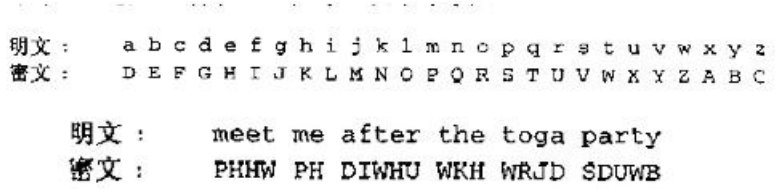
二、实验环境：

Windows10 系统
CrypTool 2.1 软件

三、实验方法（原理、流程图）

3.1 凯撒密码

凯撒密码是最早的代换密码，使用单表代换。其基本思想是：通过把字母移动一定的位数来实现加密和解密。明文中的所有字母都在字母表上向后(或向前)按照一个固定数目进行偏移后被替换成密文。例如，当偏移量是 3 的时候，所有的字母 A 将被替换成 D，B 变成 E，以此类推 X 将变成 A，Y 变成 B，Z 变成 C。位数就是凯撒密码加密和解密的密钥。



3.2 维吉尼亚密码

维吉尼亚密码引入了“密钥”的概念，即根据密钥来决定用哪一行的密表来进行替换，以此来对抗字频统计。假如以上面第一行代表明文字母，左面第一列代表密钥字母，对如下明文加密:

TO BE OR NOT TO BE THAT IS THE QUESTION

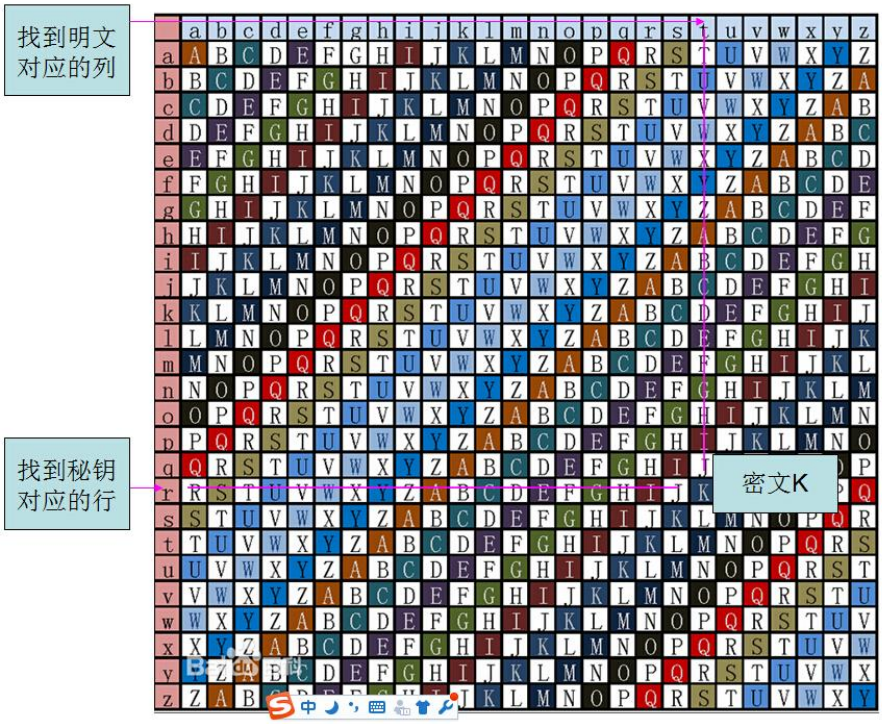
当选定 RELATIONS 作为密钥时，加密过程是：明文一个字母为 T，第一个密钥字母为 R，因此可以找到在 R 行中代替 T 的为 K，依此类推，得出对应关系如下：

密钥:RE LA TI ONS RE LA TION SR ELA TIONSREL

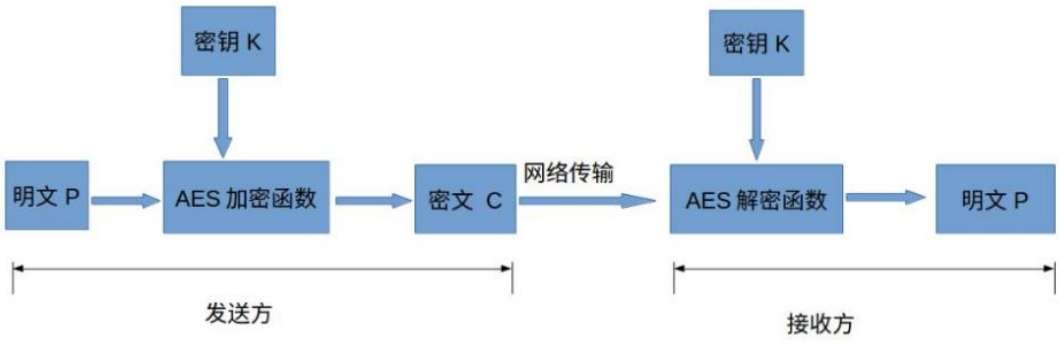
明文:TO BE OR NOT TO BE THAT　IS　THE QUESTION

密文:KS ME HZ　BBL　KS ME MPOG AJ XSE JCSFLZSY

图解加密过程:



3.3AES 加密算法



1. 明文：没有经过加密的数据。
2. 密钥 K：用来加密明文的密码，在对称加密算法中，加密与解密的密钥是相同的。密钥为接收方与发送方协商产生，但不可以直接在网络上传输，否则会导致密钥泄漏，通常是通过非对称加密算法加密密钥，然后再通过网络传输给对方，或者直接面对面商量密钥。密钥是绝对不可以泄漏的，否则会被攻击者还原密文，窃取机密数据。
3. AES 加密函数：设 AES 加密函数为 E，则 C = E(K, P),其中 P 为明文，K 为密钥，C 为密

文。也就是说，把明文 P 和密钥 K 作为加密函数的参数输入，则加密函数 E 会输出密文 C。

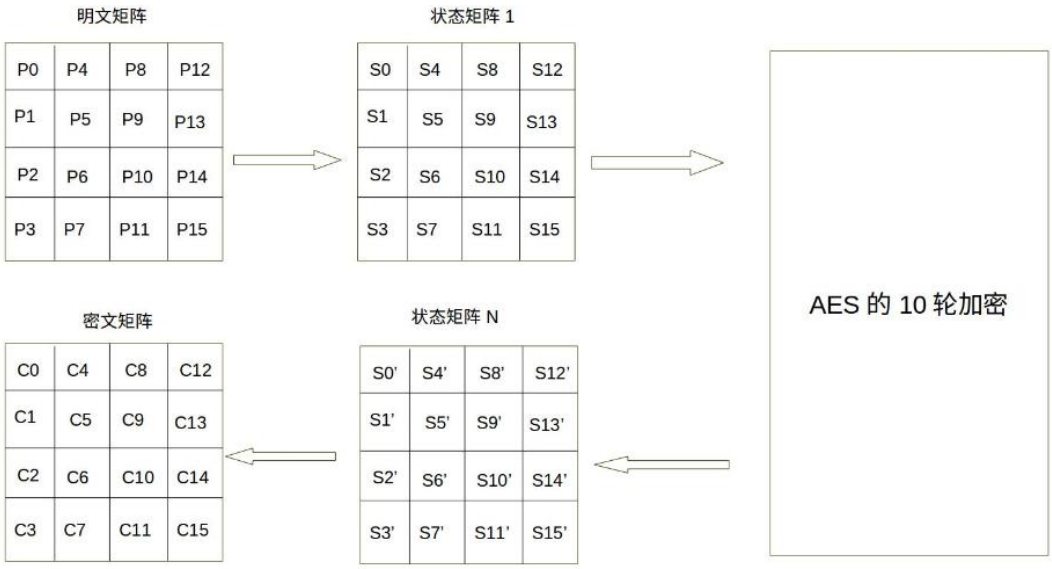
4. 密文 C：经加密函数处理后的数据。
5. AES 解密：设 AES 解密函数为 D，则 P = D(K, C),其中 C 为密文，K 为密钥，P 为明文。也就是说，把密文 C 和密钥 K 作为解密函数的参数输入，则解密函数会输出明文 P。
6. 对称加密算法：加密和解密用到的密钥是相同的，这种加密方式加密速度非常快，适合经常发送数据的场合。缺点是密钥的传输比较麻烦。
7. 非对称加密算法：加密和解密用的密钥是不同的，这种加密方式是用数学上的难解问题构造的，通常加密解密的速度比较慢，适合偶尔发送数据的场合。优点是密钥传输方便。常见的非对称加密算法为 RSA、ECC 和 ElGamal。

3.4 AES 的基本结构

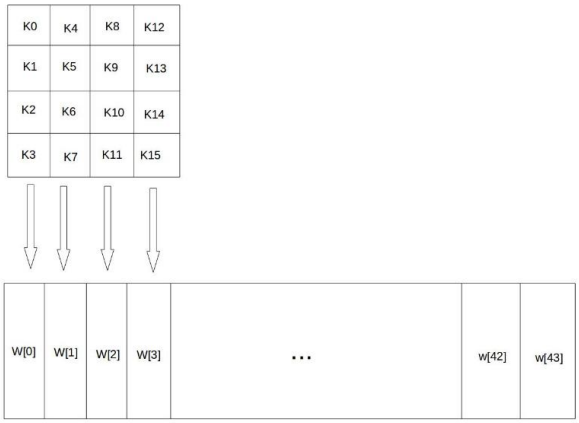
ES 为分组密码，分组密码也就是把明文分成一组一组的，每组长度相等，每次加密一组数据，直到加密完整个明文。在 AES 标准规范中，分组长度只能是 128 位，也就是说，每个分组为 16 个字节（每个字节 8 位）。密钥的长度可以使用 128 位、192 位或 256 位。密钥的长度不同，推荐加密轮数也不同，如下表所示：

AES	密钥长度 (32位比特字)	分组长度 (32位比特字)	加密轮数
AES-128	4	4	10
AES-192	6	4	12
AES-256	8	4	14
轮数在下面介绍，这里实现的是AES-128，也就是密钥的长度为128位，加密轮数为10轮。			
上面说到，AES的加密公式为C = E(K,P)，在加密函数E中，会执行一个轮函数，并且执行10次这个轮函数，这个轮函数的前9次执行的操作是一样的，只有第10次有所不同。也就是说，一个明文分组会被加密10轮。AES的核心就是实现一轮中的所有操作。			

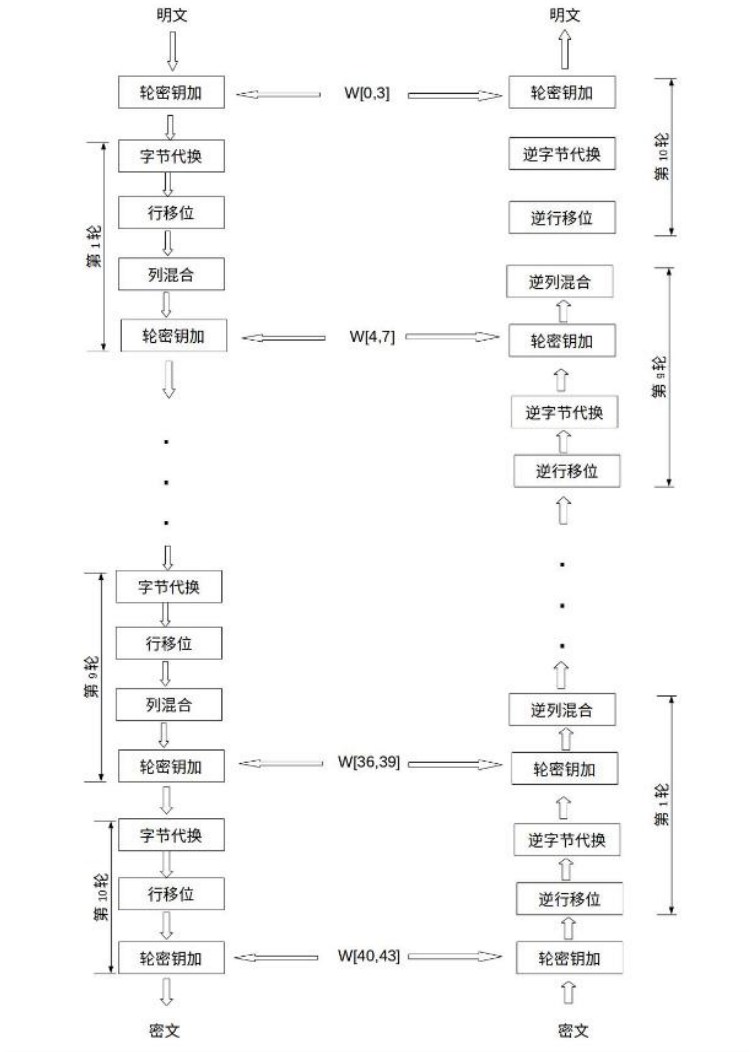
AES 的处理单位是字节，128 位的输入明文分组 P 和输入密钥 K 都被分成 16 个字节，分别记为 P = P0 P1 ⋯ P15 和 K = K0 K1 ⋯ K15。该矩阵中字节的排列顺序为从上到下、从左至右依次排列，如下图所示：



类似地，128 位密钥也是用字节为单位的矩阵表示，矩阵的每一列被称为 1 个 32 位比特字。通过密钥编排函数该密钥矩阵被扩展成一个 44 个字组成的序列 W[0],W[1], … ,W[43],该序列的前 4 个元素 W[0],W[1],W[2],W[3]是原始密钥，用于加密运算中的初始密钥加（下面介绍）；后面 40 个字分为 10 组，每组 4 个字（128 比特）分别用于 10 轮加密运算中的轮密钥加，如下图所示：



下图展示了 AES 解密过程，解密过程仍为 10 轮，每一轮的操作是加密操作的逆操作。由于 AES 的 4 个轮操作都是可逆的，因此，解密操作的一轮就是顺序执行逆行移位、逆字节代换、轮密钥加和逆列混合。同加密操作类似，最后一轮不执行逆列混合，在第 1 轮解密之前，要执行 1 次密钥加操作。



3.5 字节代换

AES 的字节代换其实就是一个简单的查表操作。AES 定义了一个 S 盒和一个逆 S 盒。状态矩阵中的元素按照下面的方式映射为一个新的字节：把该字节的高 4 位作为行值，低 4 位作为列值，取出 S 盒或者逆 S 盒中对应的行的元素作为输出。逆字节代换也就是查逆 S 盒来变换。

3.6 行移位操作

行移位是一个简单的左循环移位操作。当密钥长度为 128 比特时，状态矩阵的第 0 行左移 0 字节，第 1 行左移 1 字节，第 2 行左移 2 字节，第 3 行左移 3 字节，如下图所示：



行移位的逆变换是将状态矩阵中的每一行执行相反的移位操作，例如 AES-128 中，状态矩阵的第 0 行右移 0 字节，第 1 行右移 1 字节，第 2 行右移 2 字节，第 3 行右移 3 字节。

3.7 列混合

列混合变换是通过矩阵相乘来实现的，经行移位后的状态矩阵与固定的矩阵相乘，得到混淆后的状态矩阵。

列混合逆运算中逆变换矩阵同正变换矩阵的乘积恰好为单位矩阵。

3.8 轮密钥加

轮密钥加是将轮密钥加是将 128 位轮密钥 Ki 同状态矩阵中的数据进行逐位异或操作，如下图所示。其中，密钥 Ki 中每个字 W[4i],W[4i+1],W[4i+2],W[4i+3]为 32 位比特字，包含 4 个字节，他们的生成算法下面在下面介绍。轮密钥加过程可以看成是字逐位异或的结果，也可以看成字节级别或者位级别的操作。也就是说，可以看成 S0 S1 S2 S3 组成的 32 位字与 W[4i]的异或运算。

轮密钥加的逆运算同正向的轮密钥加运算完全一致，这是因为异或的逆操作是其自身。轮密钥加非常简单，但却能够影响 S 数组中的每一位。

3.9 密钥扩展

AES 首先将初始密钥输入到一个 44 的状态矩阵中。这个 44 矩阵的每一列的 4 个字节组成一个字，矩阵 4 列的 4 个字依次命名为 W[0]、W[1]、W[2]和 W[3]，它们构成一个以字为单位的数组 W。例如，设密钥 K 为"abcdefghijklmnop",则 K0 = ‘a’,K1 = ‘b’, K2 = ‘c’,K3 = ‘d’,W[0] = “abcd”。

接着，对 W 数组扩充 40 个新列，构成总共 44 列的扩展密钥数组。新列以如下的递归方式产生：

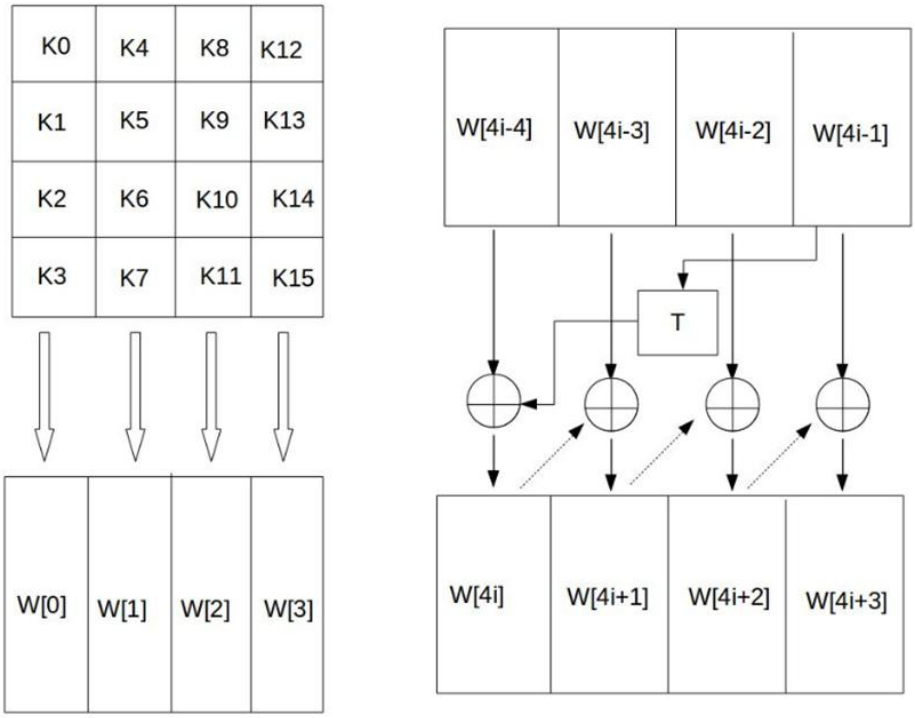
1. 如果 i 不是 4 的倍数，那么第 i 列由如下等式确定：

$$W[i]=W[i-4]\oplus W[i-1]$$

2. 如果 i 是 4 的倍数，那么第 i 列由如下等式确定：

$$W[i]=W[i-4]\oplus T(W[i-1])$$

其状态矩阵变换图如下图所示：



3.10 DES 加密

我们可以将 DES 加密过程分为以下三部分：

- 明文的加密的整体过程
- F 轮函数解析
- 密钥的形成过程

1.明文的加密整体过程

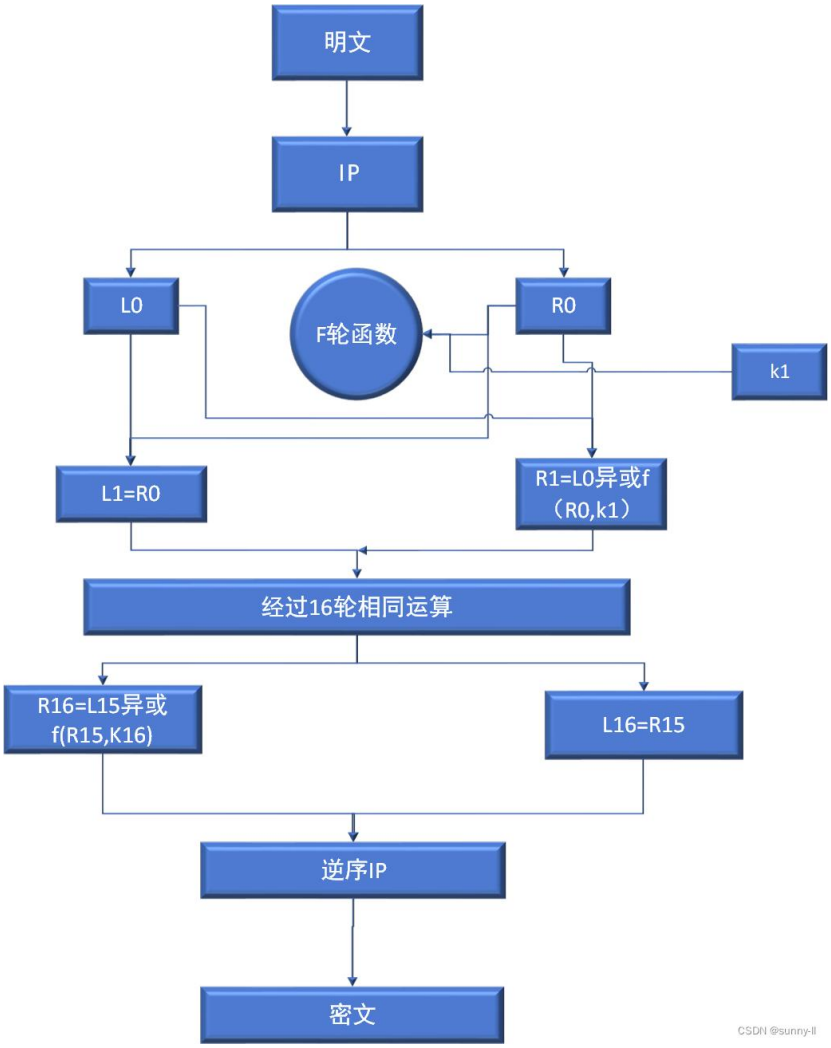
(1) 输入 64bit 的明文进行 IP 置换，分成左右两个分支各为 32bit，左边：32bitL0,右边：32bitR0

(2) 右分支：L1 = R0　左分支：引入 48bit 的密钥，R1=L0 异或　f(R0,K1)

(3) 相同的操作进行 16 次的运算循环，算出相应的，R1~R16，L0~L16

(4) 最后在进行 IP 的逆序置换，将左右两个分支再次合并为 64bit 密文

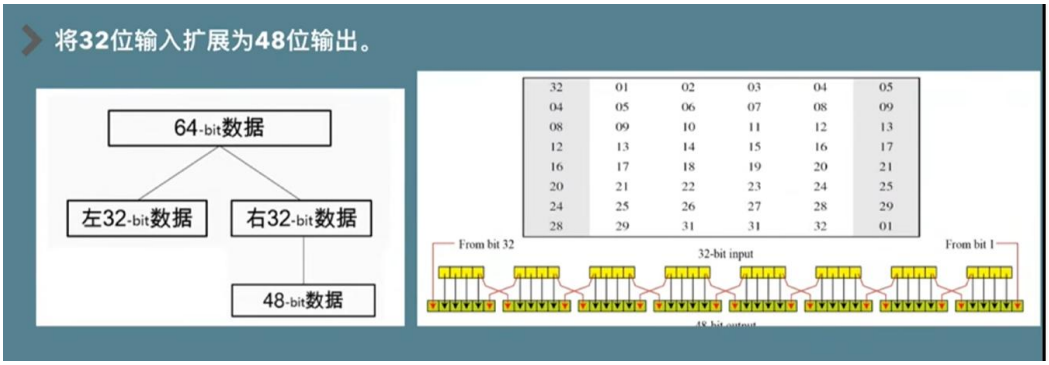
我们将上述过程画出流程图如下：



2、F 轮函数解析

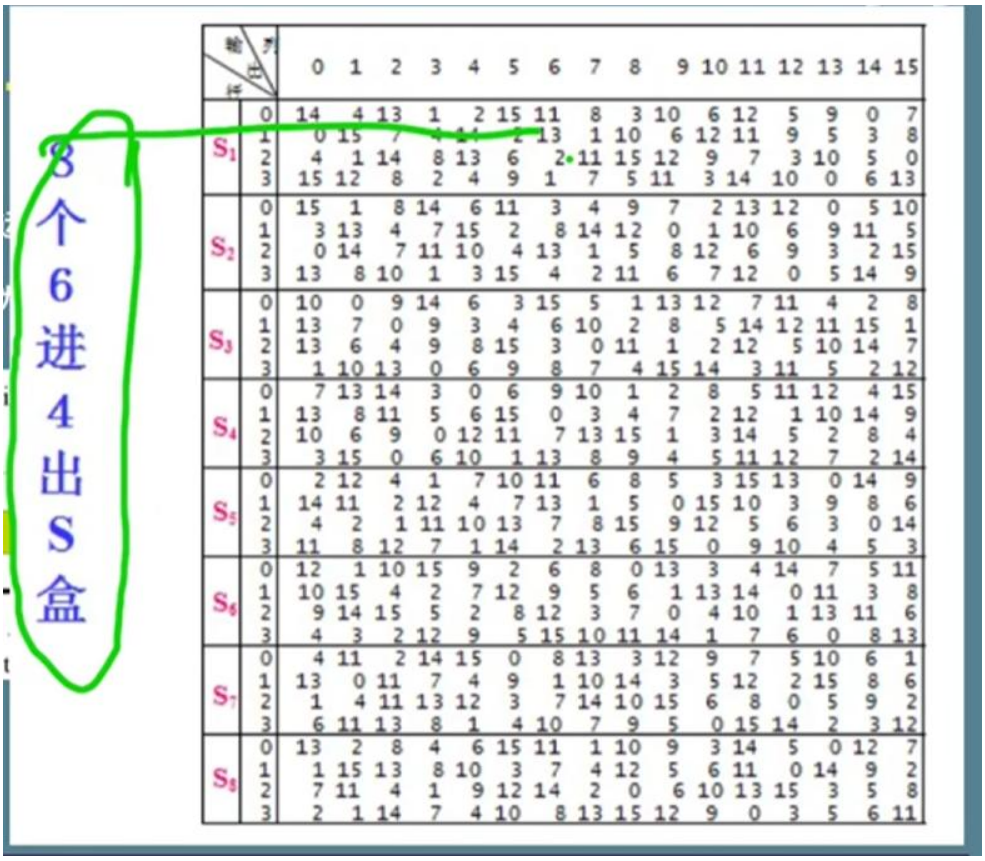
1. IP 置换

2. E 扩展：将 32bit 的 R0 扩展为 48bit 的 R0,其中扩展图为：



3. 异或：将 48bit 的 R0 于 48bit 的 K1 进行异或。

4. S 盒压缩处理 ： 大盒子里有 8 块 6bit 的小盒子，刚好容纳 48bit 的二进制数，盒子的特点是 6 进 4 出，出了盒子就变成了 32bit 的二进制数。



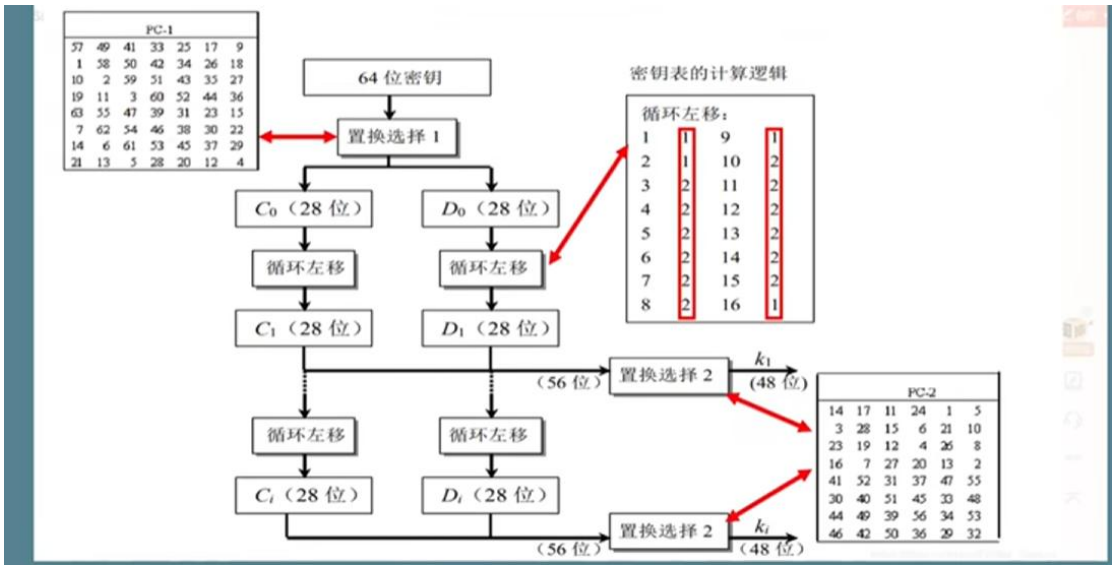
	行\列	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
S_1	0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
	1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
	2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
	3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13
S_2	0	15	1	8	14	6	11	3	4	9	7	2	13	12	0	5	10
	1	3	13	4	7	15	2	8	14	12	0	1	10	6	9	11	5
	2	0	14	7	11	10	4	13	1	5	8	12	6	9	3	2	15
	3	13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9
S_3	0	10	0	9	14	6	3	15	5	1	13	12	7	11	4	2	8
	1	13	7	0	9	3	4	6	10	2	8	5	14	12	11	15	1
	2	13	6	4	9	8	15	3	0	11	1	2	12	5	10	14	7
	3	1	10	13	0	6	9	8	7	4	15	14	3	11	5	2	12
S_4	0	7	13	14	3	0	6	9	10	1	2	8	5	11	12	4	15
	1	13	8	11	5	6	15	0	3	4	7	2	12	1	10	14	9
	2	10	6	9	0	12	11	7	13	15	1	3	14	5	2	8	4
	3	3	15	0	6	10	1	13	8	9	4	5	11	12	7	2	14
S_5	0	2	12	4	1	7	10	11	6	8	5	3	15	13	0	14	9
	1	14	11	2	12	4	7	13	1	5	0	15	10	3	9	8	6
	2	4	2	1	11	10	13	7	8	15	9	12	5	6	3	0	14
	3	11	8	12	7	1	14	2	13	6	15	0	9	10	4	5	3
S_6	0	12	1	10	15	9	2	6	8	0	13	3	4	14	7	5	11
	1	10	15	4	2	7	12	9	5	6	1	13	14	0	11	3	8
	2	9	14	15	5	2	8	12	3	7	0	4	10	1	13	11	6
	3	4	3	2	12	9	5	15	10	11	14	1	7	6	0	8	13
S_7	0	4	11	2	14	15	0	8	13	3	12	9	7	5	10	6	1
	1	13	0	11	7	4	9	1	10	14	3	5	12	2	15	8	6
	2	1	4	11	13	12	3	7	14	10	15	6	8	0	5	9	2
	3	6	11	13	8	1	4	10	7	9	5	0	15	14	2	3	12
S_8	0	13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7
	1	1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2
	2	7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8
	3	2	1	14	7	4	10	8	13	15	12	9	0	3	5	6	11

（5）IP 逆序置换

3.密钥的形成过程

（1）密钥原本为 64bit ,去掉 8 位校验位，剩余 56 位参与运算。

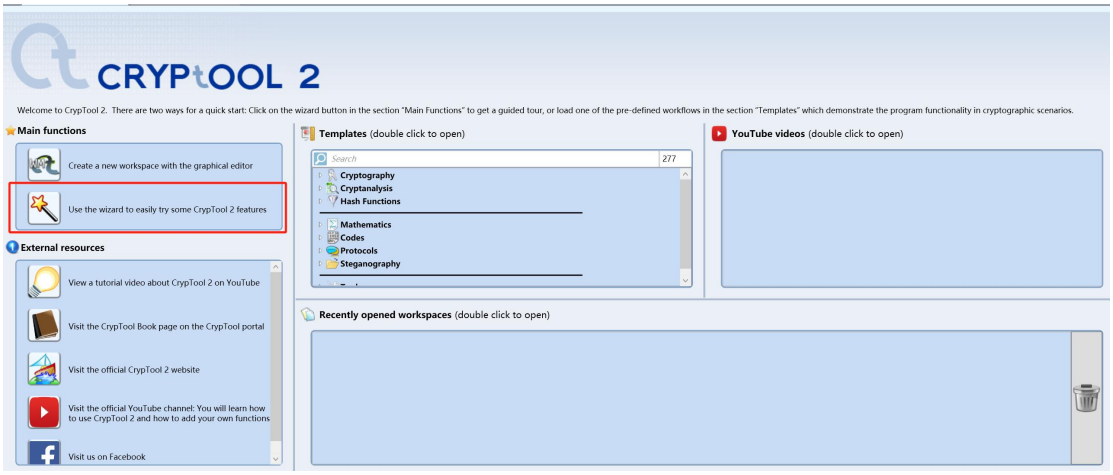
（2）按照交换规则，生成 16 位 48bit 的轮密钥。



四、实验过程、步骤及内容

4.1 使用 Wizard 来测试密码：

在 CRYPtOOL2 中，我们可以使用 Wizard 中来测试加解密方法，点击主页中的 Wizard(魔法棒样式)；



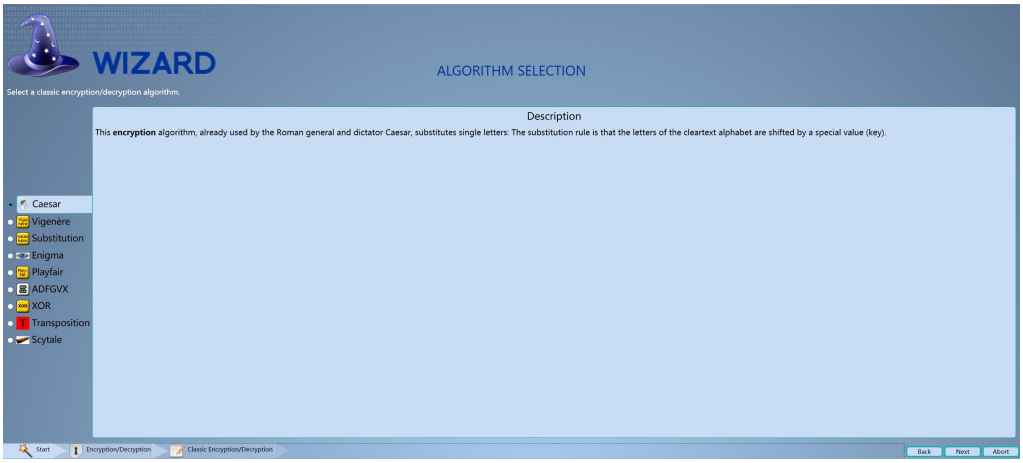
接下来我们会进入到以下界面，点击左侧第一栏，我们可以选择自己想要测试的加解密密码：



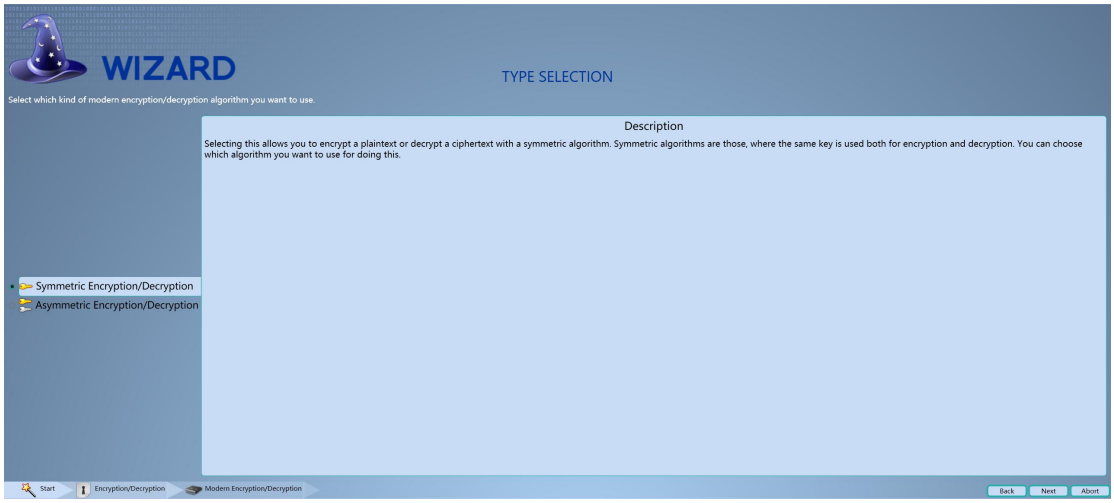
点击进入后我们可以看到左侧侧边栏存在着两个选项，上面一个选项是古典密码的加密与解密，下面一个是现代密码的加密和解密。

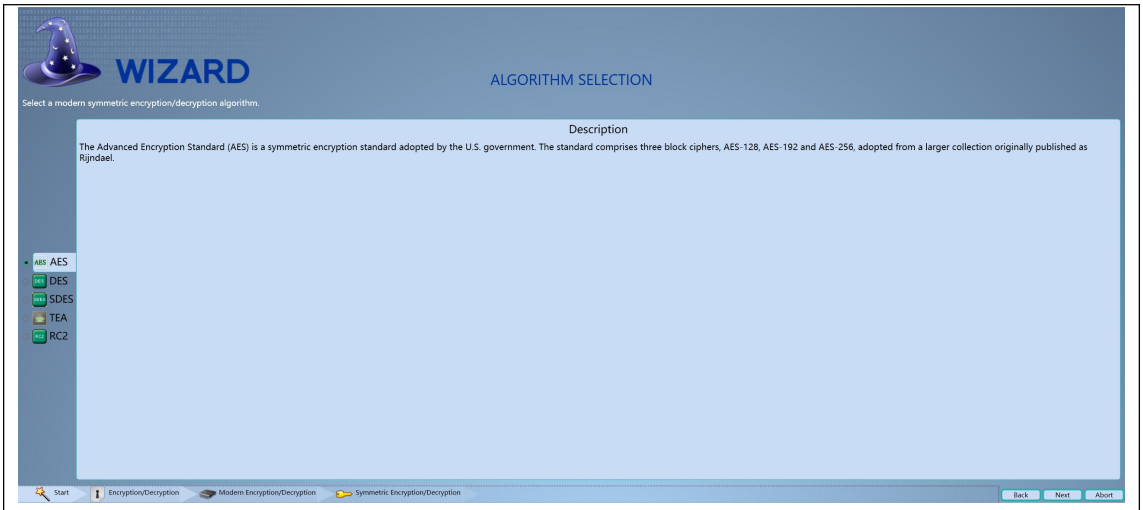


点击古典密码，我们可以看到侧边栏存在多种古典加密算法，在本次实验中我们仅仅只需要测试凯撒密码和维吉尼亚密码。



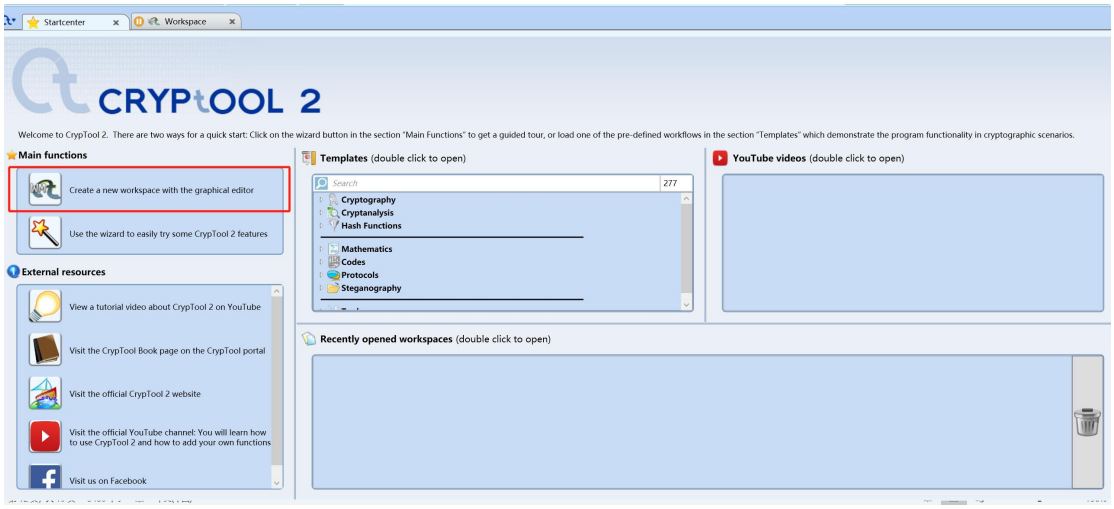
点击现代密码后，我们可以看到侧边栏存在对称加密和非对称加密两种选项，我们选择其中的对称加密算法，可以看到左侧侧边栏存在许多对称加密算法，然后我们需要完成 AES 和 DES 加解密算法的测试。





4.2 使用 Graphical Editor 来实现凯撒密码的加解密

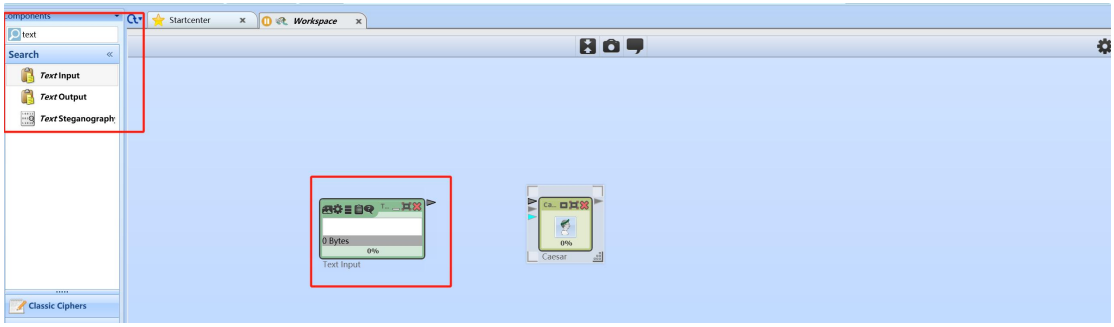
1.点击 star center 中的 workspace 打开一个工作区：



2. 在工作区中我们选择古典加密器中的凯撒密码进行加解密，并将其拖动至工作区：



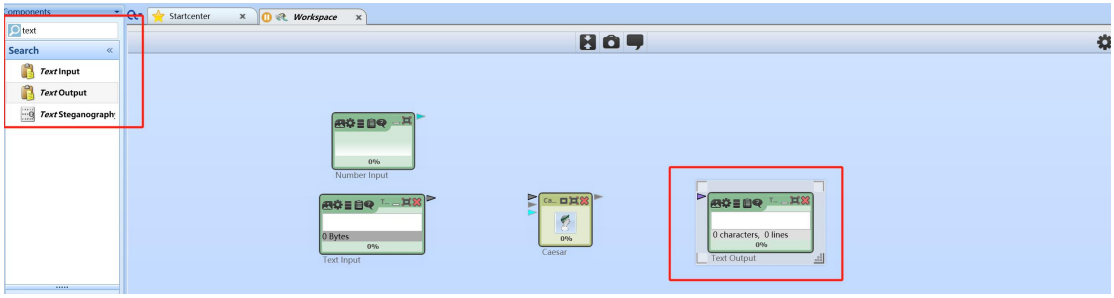
3. 在工作区中选择 Text Input，用来存放我们所需要输入的密文，并将其拖动至工作区。



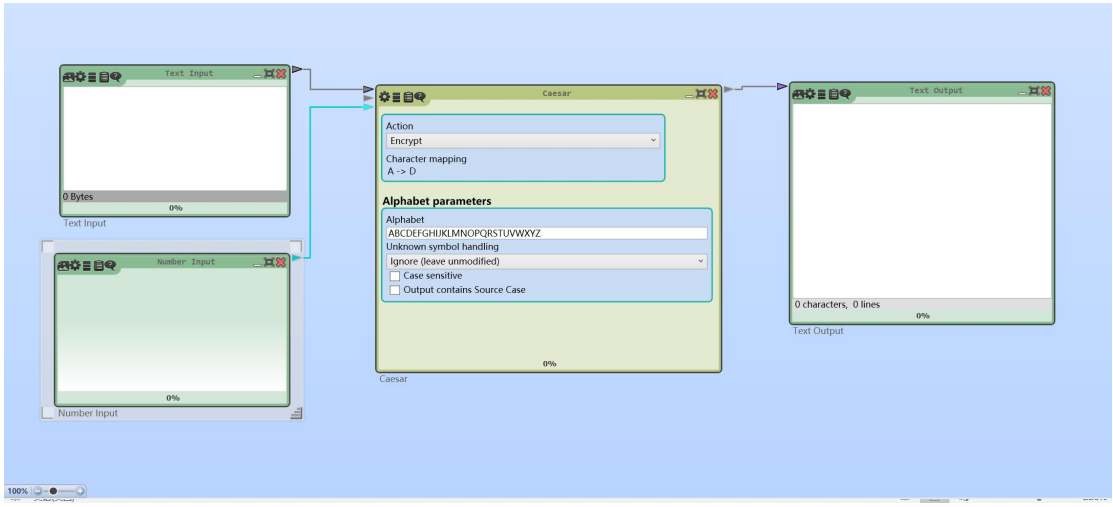
4.在工作区中选择 Number input,表示我们所需要输入的密钥，并将其拖动至工作区。



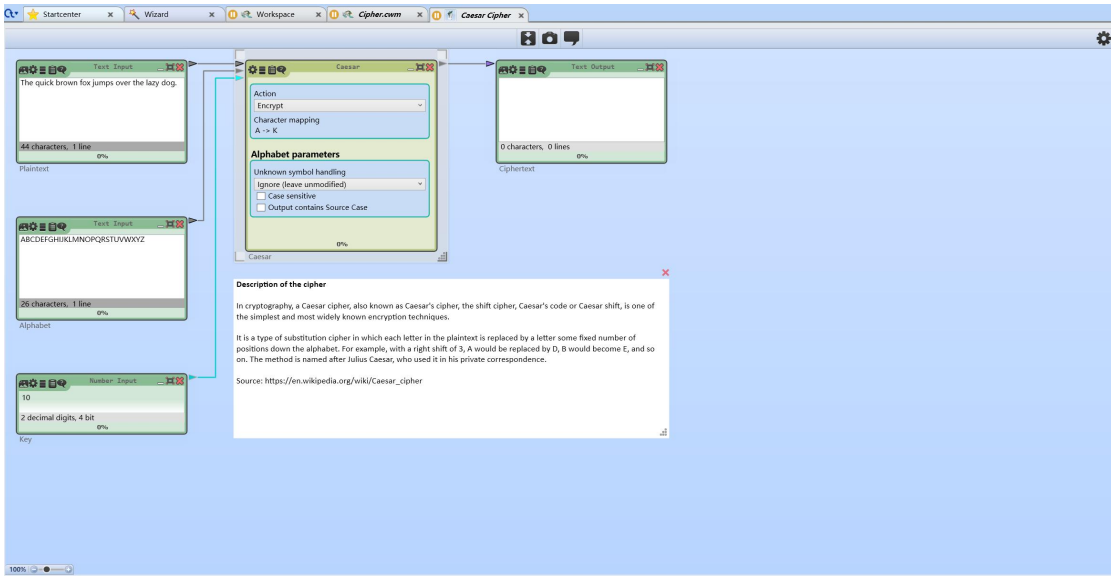
5.在工具栏中选择 text output 用来存放我们所需要输出的字符串，并将其拖动至工作区内。



6.最终我们可以得到如下图所示的工作区样例。



7.我们也可以在 Templates 中选择前人已经做好的样例，然后在此基础上对样例进行修改。



它与我们所作样例的不同点为它的字母表为从外界输入的。

五、实验数据（现象）处理分析

5.1 使用 Wizard 功能进行古典密码验证

运行 CrypTool2 软件，选择 Wizard，随后选择古典密码加解密（Classic Encryption/Decryption），软件提供多种加密方法的实现，界面如图 5.1 所示。

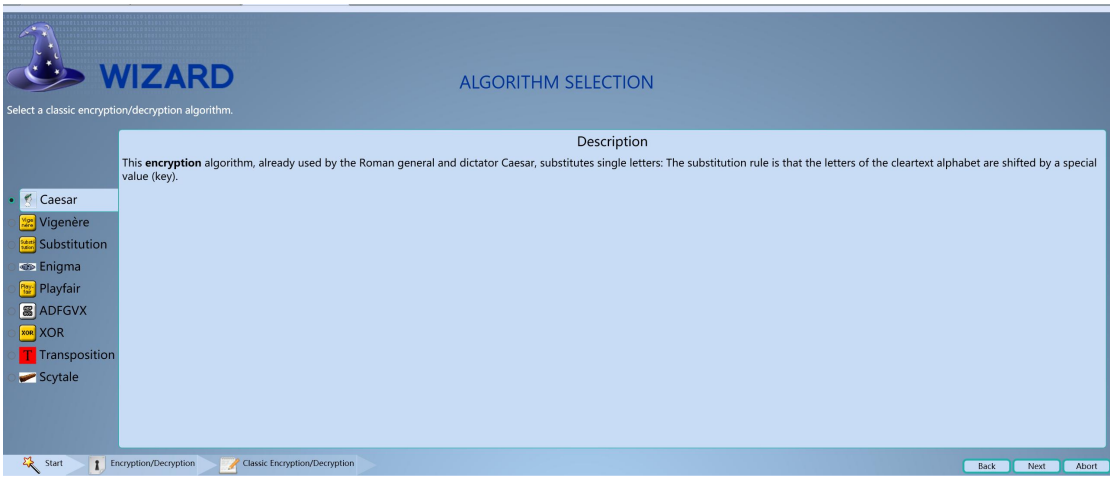


图 5.1 CrypTool Wizard 界面

分别选择 Caesar 密码、Vigenere 密码、Substitution、PlayFair 进行实验。

（1）Caesar 密码

选择加密模式，设置密文为“226001624Gukaiqi”，密钥为 4，如图 5.2 所示。加密结果如图 5.3 所示，密文为：“226001624Jxndltl”。

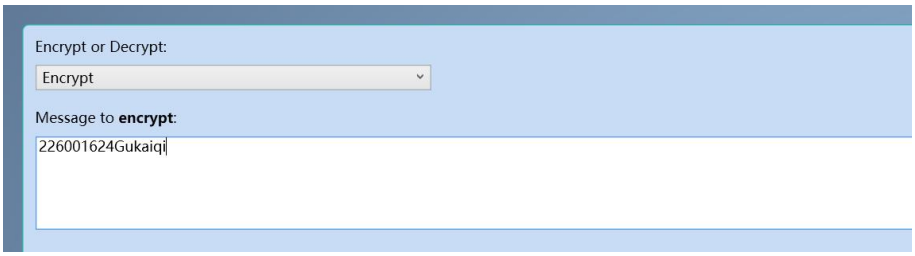


图 5.2 设置明文和密钥



图 5.3 使用 Caesar 密码加密

可以看到，由于凯撒密码进行的是偏移加密，明文字母与密文字母之间具有一一对应关系，虽然密文不能直接被理解，但是可以通过一些常用的明文字符进行含义推断，例如图5.3中的密文，字符‘l’呈现一定的规律，若文本量较大，则可以进行对应关系的推断。

将密文“226001624Jxndltl”和密钥4作为输入给与解密程序，结果如图5.4所示，可以看到完成了解密，解密结果与明文一致。



图 5.4 使用 Caesar 密码解密

（2）Vigenere 密码

选择加密模式，设置密文为“226001624Gukaiqi”，密钥为“nicejob”，执行加密后，密文为：“226001624Tcmerej”。结果如图5.5所示



图 5.5 使用 Vigenere 密码加密

将密文“226001624Tcmerej”和密钥“nicejob”作为输入给与解密程序，结果如图5.4所示，可以看到完成了解密，解密结果与明文一致。



图 5.6 使用 Vigenere 密码解密

将相比于凯撒密码，Vigenere 密码的偏移量不唯一，由密钥给出具体的偏移量，更难找到统计规律，但并没有完全消除统计规律的存在。

（3）Substitution

选择加密模式，设置密文为“226001624Gukaiqi”，密钥为“Keyword”，执行加密后，密文为：“226001624dthUfnf”。结果如图 5.7 所示

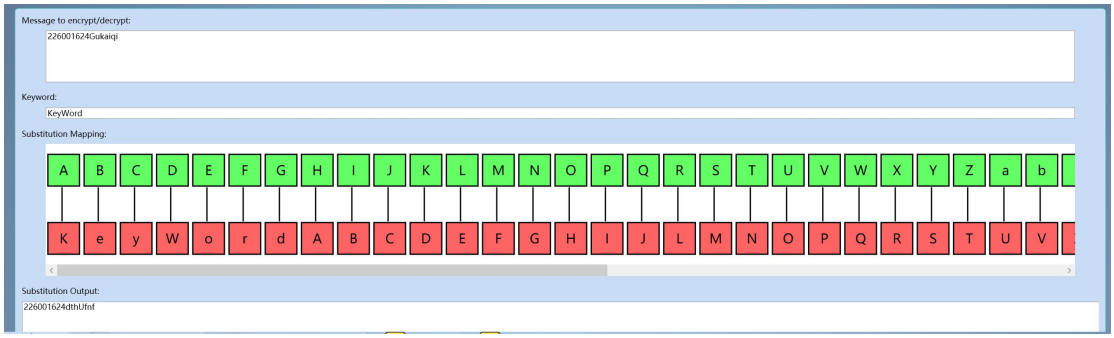


图 5.7 使用 Substitution 密码加密

将密文“226001624dthUfnf”和密钥“Keyword”作为输入给与解密程序，结果如图 5.8 所示，可以看到完成了解密，解密结果与明文一致。

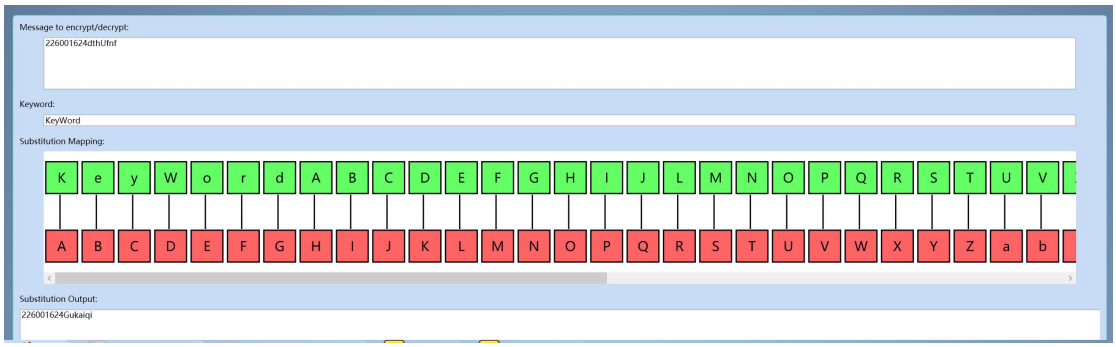


图 5.8 使用 Substitution 密码解密

（4）使用 PlayFair 对同上类型的明文进行加密解密操作。

选择加密模式，设置密文为“GUKAIQIPLAYING”，密钥为“GKQNB”，执行加密后，密文为：“KRIEFTKOOBAOMH”。结果如图 5.9 所示



图 5.9 使用 PlayFair 密码加密

将密文“KRIEFTKOOBAOMH”和密钥“GKQNB”作为输入给与解密程序，结果如图 5.10 所示，可以看到完成了解密，解密结果与明文一致。

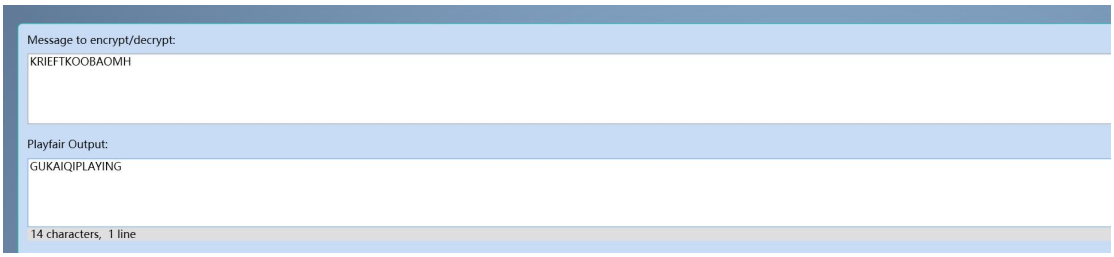


图 5.10 使用 PlayFair 密码解密

5.2 使用 Wizard 功能进行现代密码验证

选择古典密码加解密（Modern Encryption/Decryption），软件提供 DES、 AES、SDES、TEA、RC2 等方法的实现。

（1）DES 对称加解密

选择加密模式，设置密钥与密文“GukaiqiLearnDES”，如图 5.11 所示。执行加密操作，结果如图 5.12 所示，DES 的输出为：“E4 AE AC 03 AD 72 6A 50 EB 8D DD 00 DC 44 B8 91”。

对于解密操作，选择解密模式，给与密钥与上一步中 DES 输出的密文，进行解密操作，结果如图 5.13 所示，可以看到解密结果与明文一致，实验结果符合预期。

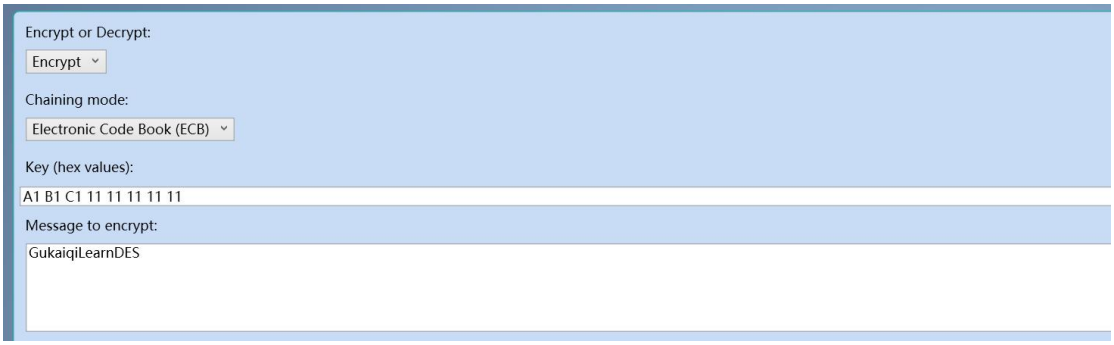


图 5.11 DES 加密操作的明文与密钥

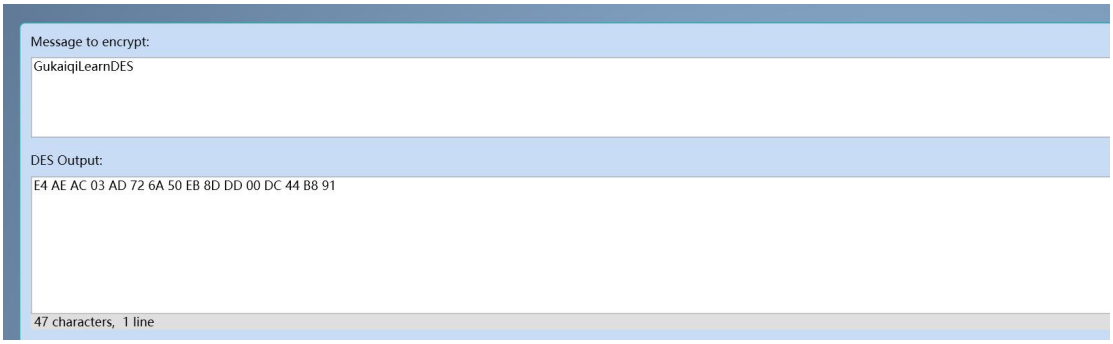


图 5.12 DES 加密结果

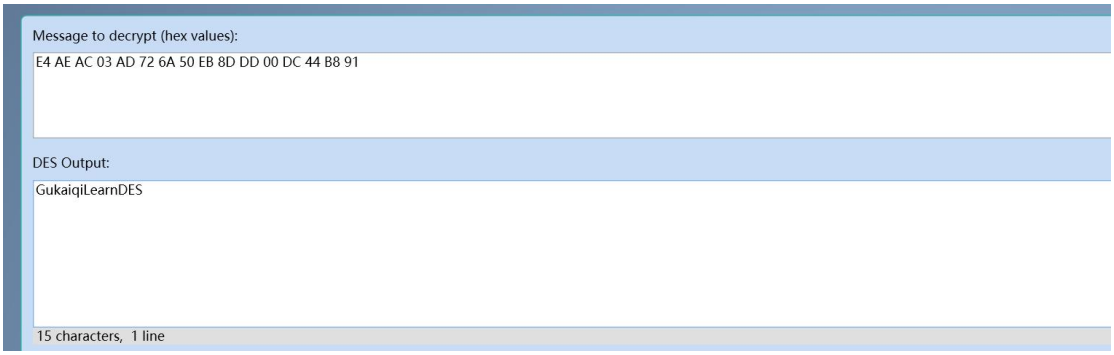


图 5.13　DES 解密结果

（2）AES 对称加解密

选择加密模式，设置密钥与密文“GukaiqiLearnAES”，其中密钥长度设置为 128，如图 5.14 所示。执行加密操作，结果如图 5.15 所示，AES 的输出为：“BF A6 8F FF D0 57 2D 13 E8 77 F1 29 3B 7B 91 60”。

对于解密操作，选择解密模式，给与密钥与上一步中 AES 输出的密文，进行解密操作，结果如图 5.16 所示，可以看到解密结果与明文一致，实验结果符合预期。

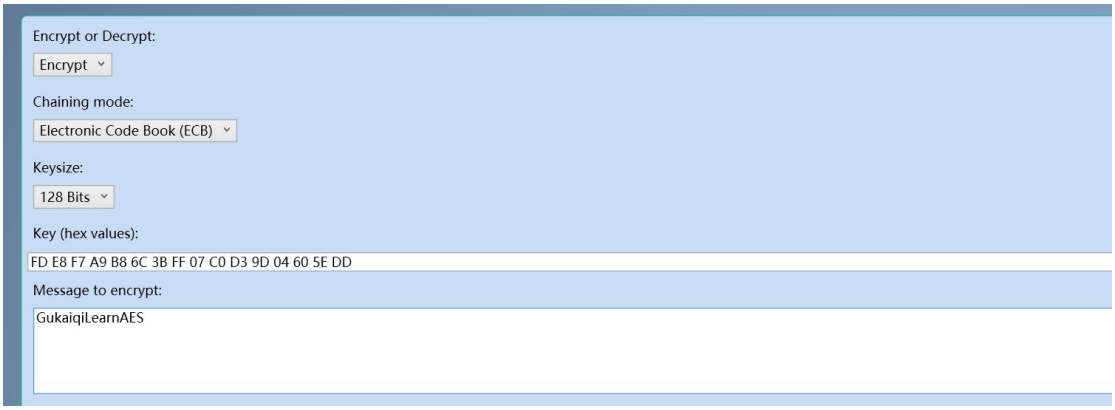


图 5.14　AES 加密操作的明文与密钥

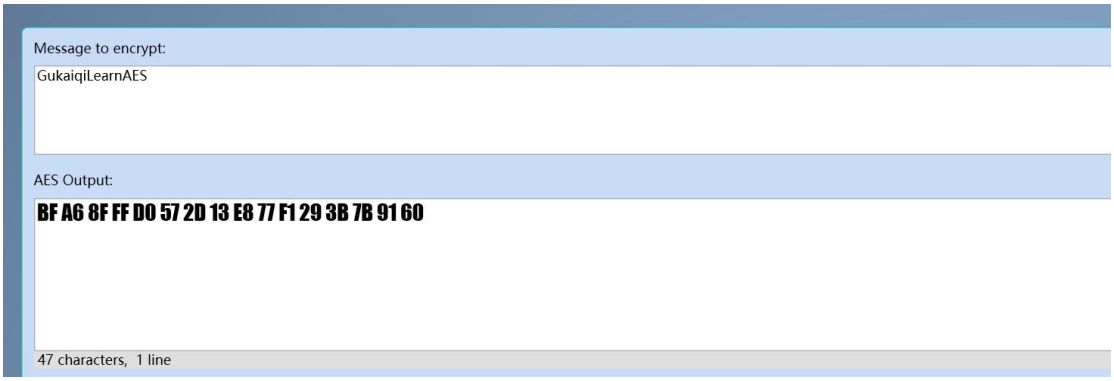


图 5.15　AES 加密结果

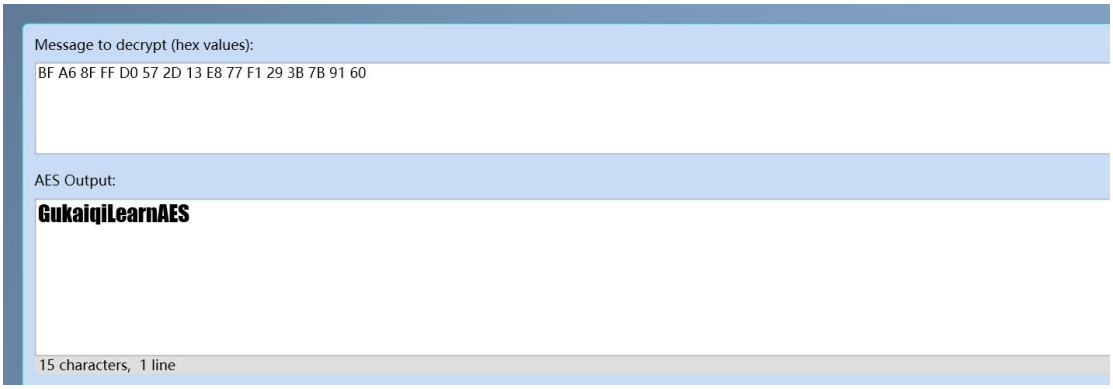


图 5.13　AES 解密结果

5.3 使用 Graphical Editor 实现 Caesar 密码的加解密

使用 Graphical Editor，悬着 Caesar 密码模块，将其设置为加密模式，为了将明文和密钥进行输入，需要使用 Text Input 模块和 Number Input 模块，为了接收加密好的密文，需要一个 Text Output 模块，并将模块数据流进行连接，如图 5.14 所示。

完成模块的设置后，点击运行即可完成加密过程，结果即如图 5.14 所示，可以看到密文成功生成。

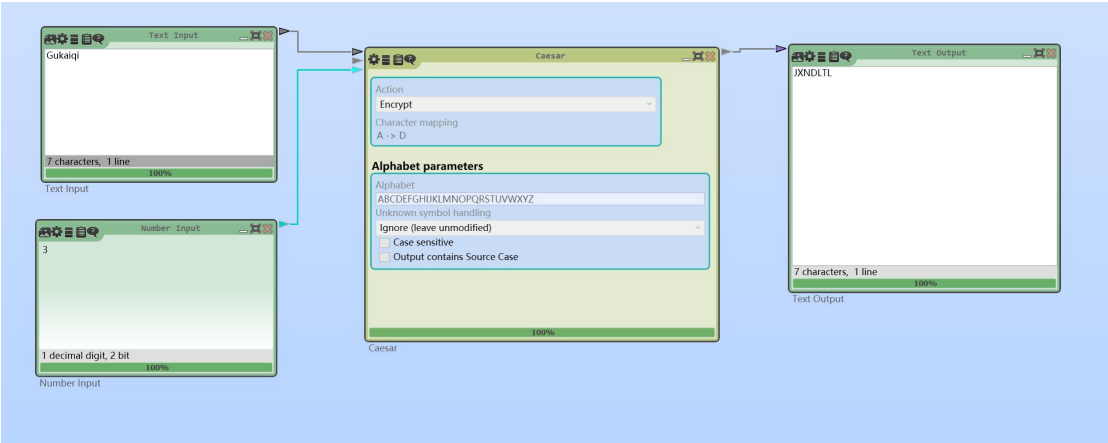


图 5.14 使用可视化编辑器试验凯撒密码

为了实现解密并验证加密结果的正确性，将凯撒密码模块设置为解密模式，将密文置于 Text Input 模块中，执行解密过程，结果如图 5.15 所示，可以看到完成了解密，且解密的结果与明文一致，符合实验预期。

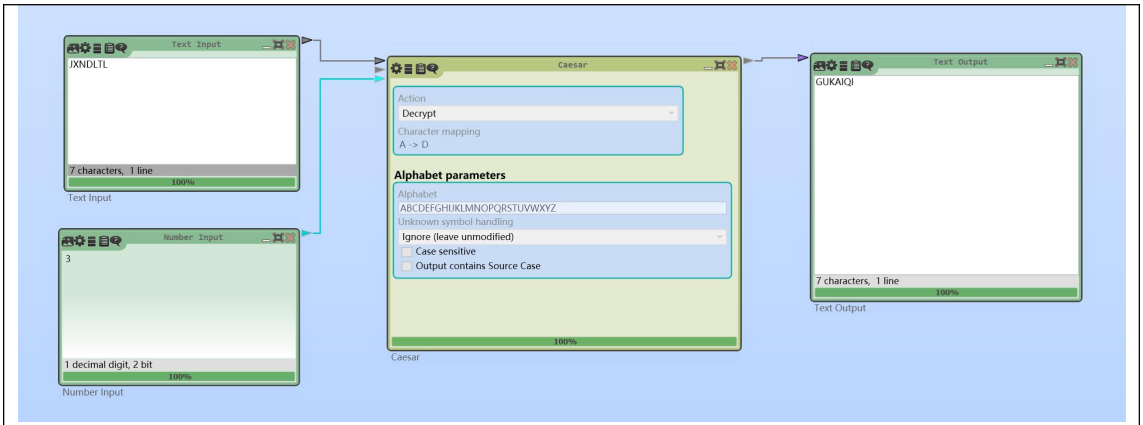


图 5.15 凯撒密码解密

5.4 使用 Graphical Editor 构建多重加解密结构

考虑到将多种加密方法进行组合往往能够得到更为复杂的加密结果，因此多重加密也常被应用于各类加密场景中。

对于基于分组的对称加密算法，经过多轮迭代最终输出一组二进制数位，可以将其使用十六进制进行表示并将其视为字符串形式，对该字符串再进行一次古典加密（例如 Vigenere 加密）即可得到一个再次加密的字符串，其中加密表由密钥给出，密钥结合了 DES 与 Vigenere 中涉及的密钥。整体流程如下图所示。

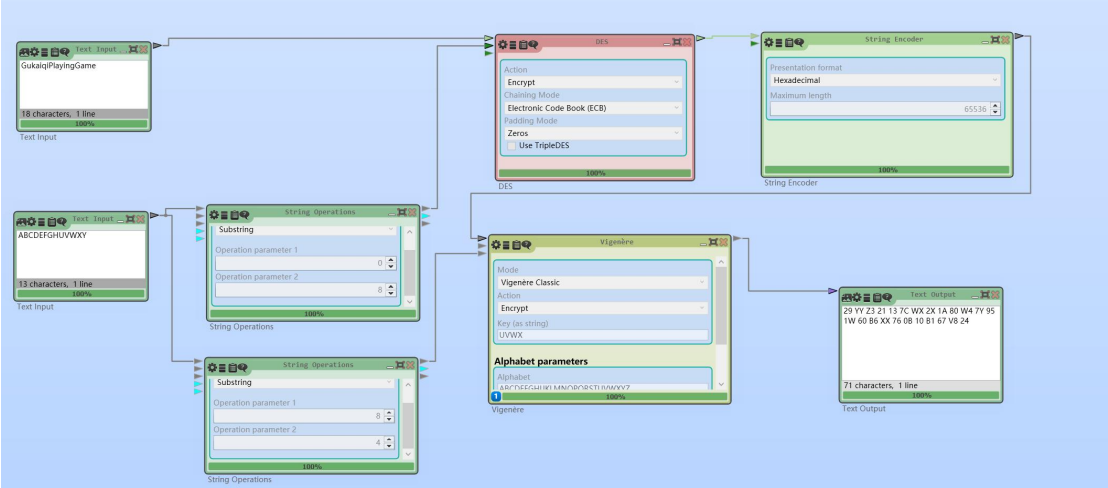


图 5.16 多重加密结构

输入的明文为“GukaiqiPlayingGame”，密钥为“ABCDEFGHUVWXY”，最终加密得到的密文为“29 YY Z3 21 13 7C WX 2X 1A 80 W4 7Y 95 1W 60 B6 XX 76 0B 10 B1 67 V8 24”。

为验证其加密的可逆性，设计其解密结构，如图 5.17 所示，可以看到解密结构能够还原出明文，符合实验预期。过程中需要注意数据流的编码方式，使用 StringDecoder 将十六进制字符串转换为对应的十六进制数才能够传入 DES 进行进一步的解密，解密结果也为数值表示法，实际上字符串是通过 ASCII 值进行表示的，因此需要送入一个 Encoder 将 DES 的

输出结果转换回字符串即可显示明文。

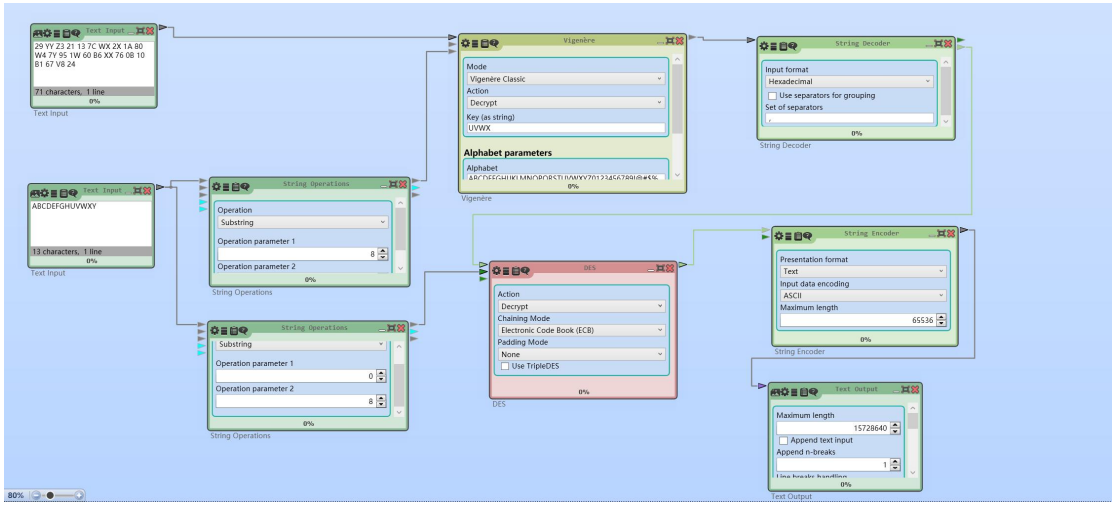


图 5.17 多重解密结构

六、实验结论

通过本次对称加密算法实验，我深入理解了 CrypTool 软件的基本功能，并成功完成了四种对称加密算法的加解密过程，包括凯撒密码、维吉尼亚密码、AES 和 DES 算法。在实验过程中，通过 CrypTool 的 Wizard 功能，验证了各加密算法的加解密过程，并通过 Graphical Editor 实现了凯撒密码的加解密。通过实验，我不仅熟悉了古典加密算法的基本原理，也掌握了现代对称加密算法如 AES 和 DES 的加密过程和密钥管理方法。

在实际操作中，凯撒密码和维吉尼亚密码等古典密码算法相对简单，通过 Shift 和 Key 表的替代即可实现加解密。而 AES 和 DES 作为现代加密算法，提供了更高的安全性，但其复杂性和密钥管理要求也更高。实验使我进一步了解了对称加密在数据保护中的重要性，以及如何在实际应用中合理选择加密算法。

总的来说，本次实验不仅加深了我对加密算法原理的理解，还提高了我在实际应用中使用加密工具的能力，尤其是通过 CrypTool 进行密码学操作的技能。

七、教师批阅意见：   成绩评定：   教师签字： 年　　月　　日
八、备注：